SAVEETHA SCHOOL OF ENGINEERING
Affliated to AICTE | IET-UK Accreditation
Engineer to Excel

SAVEETHA
INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Novel HMAC-Based Software Update Integrity Verification Platform

Capstone Project

Guided By:
Dr. Sivasankar

Presented By:
Narmatha J (192521341)
D. S. Tharuniya Priya (192511317)
Lakshumipriya O S (192521197)
P. Monisha (1965260159)

INTRODUCTION

- Software updates must be verified before they are accepted.
- Attackers may modify or replace update files during distribution.
- A modified update may contain malicious code.
- Our system uses HMAC-SHA-256 to verify package integrity and authenticity.
- It records verification results and alerts administrators when tampering is detected.



Engineer to Excel

PROBLEM STATEMENT

- Users install updates without checking them.
- A fake or changed update can harm the computer.
- A normal hash can be changed by an attacker too.
- There may be no convenient, centralized way to verify update integrity and track tampering attempts.
- We need a safe system to verify every update.



Engineer to Excel

OBJECTIVES

- To store all software packages in one place.
- To generate hash and HMAC for each update.
- To check the package before it is installed.
- To reject packages that fail integrity verification before they are accepted for installation.
- To save the update history and show tamper alerts.



Literature Survey and Research Gap

Literature Survey:

S. No.	Author & Year	Key Findings
1	Usman et al. (2023)	Studied software supply-chain attacks and malicious binary modifications.
2	Reichert & Obelheiro (2024)	Reviewed software supply-chain security challenges and research gaps.
3	Nahum et al. (2024)	Proposed collaborative integrity verification for open-source libraries.

Research Gap:

- Limited integration of package verification and centralized tamper-history tracking in a lightweight platform.
- Proposed system uses HMAC-SHA-256 to verify packages and log verification failures.

PROPOSED SYSTEM

- Admin uploads the software update package.
- Backend generates the HMAC using a secret key.
- Trusted HMAC is stored securely.
- System compares the generated and stored HMAC.
- Matching packages are verified.
- Modified packages are rejected, and alerts are recorded.

Advantages:

- Detects unauthorized changes.
- Verifies package integrity and authenticity.
- Protects the secret key on the backend.

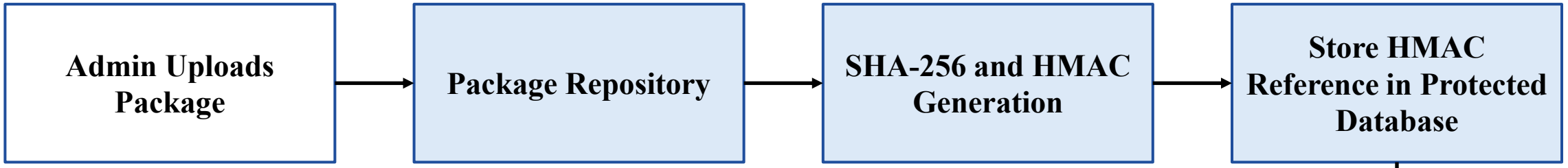
Engineer to Excel

TOOLS AND TECHNOLOGIES

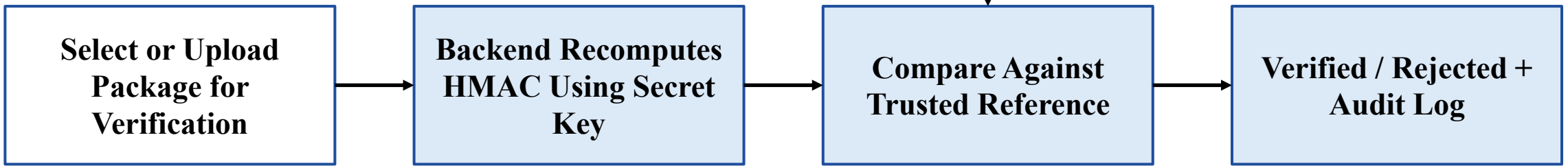
- Programming Language: Python
- Backend Framework: Flask
- Frontend: HTML, CSS, Bootstrap
- Database: SQLite
- Cryptographic Libraries: Python hashlib and hmac
- Development Environment: Visual Studio Code

SYSTEM ARCHITECTURE

SERVER SIDE (UPLOAD FLOW)



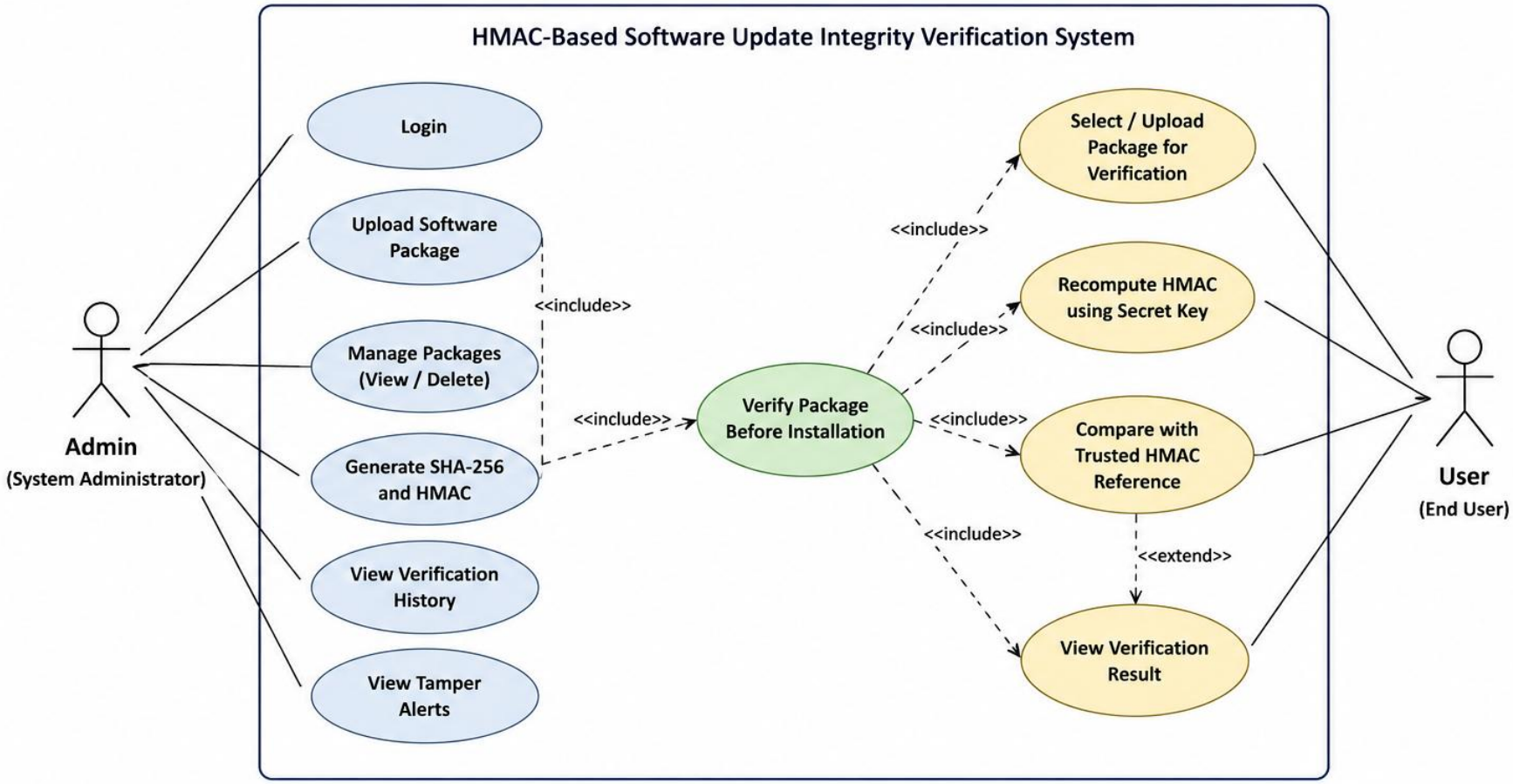
CLIENT / VERIFICATION FLOW



Secret key is kept on the trusted backend (environment variable), separate from the reference HMAC database.

Engineer to Excel

UML DIAGRAM



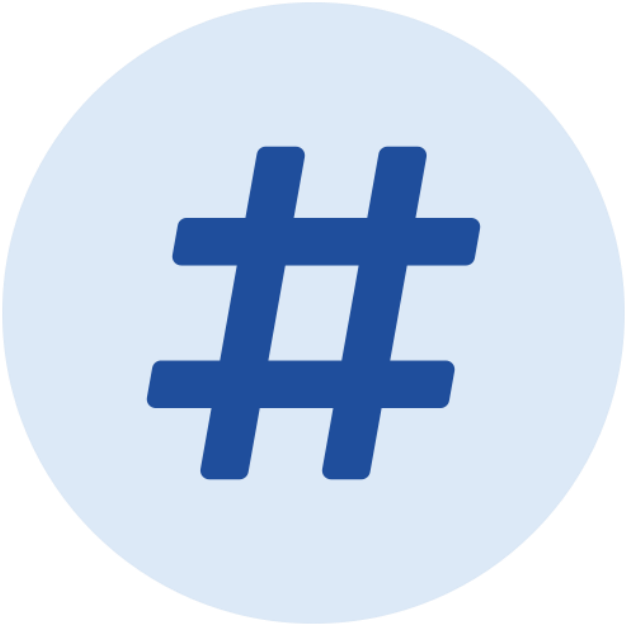
Module 1 – Software Package Repository

- This module stores all the software packages.
- Admin can upload new update files.
- Each package has a name and version number.
- Old versions are also kept safely.
- Upload validation checks the allowed file types and a maximum file size.
- Users download updates from here.



Module 2 – Update Hash and HMAC Generation

- This is the main part of the project.
- It creates a hash of the package using SHA-256.
- Then it makes an HMAC using a secret key.
- The HMAC reference is stored in the database separately from the package file.
- The secret key is kept securely on the backend and is never exposed to clients.



Module 3 – Package Verification Before Installation

- This module checks the package before installing.
- It makes the HMAC again for the downloaded file.
- Then it compares it with the saved HMAC.
- If the computed HMAC matches the trusted reference, the package is marked as verified.
- If it does not match, the package is rejected and a tampering alert is recorded.



Module 4 – Update History and Tamper Alerts

- It saves the details of every update.
- It shows the result as Verified or Tampered.
- If a file is changed, it shows an alert.
- The admin can see the full history.
- This helps administrators investigate verification failures and possible tampering attempts.



HMAC Algorithm Used

HMAC (Hash-based Message Authentication Code)

HMAC-SHA-256 uses a cryptographic hash function and a shared secret key to verify the integrity and authenticity of a message or software package.

The verification result is trustworthy only when the secret key and reference HMAC are protected.



Formula:

$$\mathrm{HMAC}(K, M) = H((K \oplus \mathrm{opad}) \,||\, H((K \oplus \mathrm{ipad}) \,||\, M))$$

- K = Secret key
- M = Software package (message)
- H = SHA-256 hash function

IMPLEMENTATION AND RESULTS

Initial implementation:

- Package upload interface
- HMAC generation process
- Verification workflow
- Result display and history logging

Planned Test Cases:

- Original package
- Modified package
- Incorrect trusted HMAC
- Verification failure

Expected Behaviour:

Valid packages pass verification; mismatched packages are rejected.

Engineer to Excel

CONCLUSION

- This project verifies software package integrity using HMAC-SHA-256.
- It compares each package against a trusted HMAC reference.
- Packages that fail verification are rejected before acceptance.
- The system maintains update history and records tampering alerts.
- It provides an additional layer of protection against unauthorized package modification.

